EXPERIMENT 3

Q. Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE Assignment3_DB;
USE Assignment3_DB;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(100) NOT NULL
);

CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(100) NOT NULL,
   price INT CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT PRIMARY KEY,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT CHECK (qty_sold > 0),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
   FOREIGN KEY (item_id) REFERENCES Item(item_id));
```

2. Insert around 10 records in each of the tables.

```
INSERT INTO Customer VALUES
(1,'Amit'),
(2,'Neha'),
(3,'Rahul'),
(4,'Sneha'),
(5,'Rohit'),
(6,'Priya'),
(7,'Karan'),
(8,'Pooja'),
(9,'Ankit'),
(10,'Simran');

INSERT INTO Item VALUES
(101,'Pen',50),
(102,'Notebook',150),
(103,'Bag',1200),
(104,'Bottle',300),
(105,'Shoes',2000),
(106,'Mouse',400),
(107,'Keyboard',800),
(108,'Calculator',500),
(109,'File',100),
(110,'Charger',600);

INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,2),
(1002,CURDATE(),2,103,1),
(1003,CURDATE(),5,105,1),
(1004,CURDATE()-INTERVAL 1 DAY,3,104,3),
```

```
(1005,CURDATE()-INTERVAL 2 DAY,5,106,2),
(1006,CURDATE(),4,108,1),
(1007,CURDATE()-INTERVAL 3 DAY,6,110,1),
(1008,CURDATE(),5,102,4),
(1009,CURDATE()-INTERVAL 6 DAY,7,107,1),
(1010,CURDATE(),8,109,5);
```

3. Bills for today with customer name & item number

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	cust_name	item_id
1001	Amit	101
1002	Neha	103
1003	Rohit	105
1006	Sneha	108
1008	Rohit	102
1010	Pooja	109

4. Total bill details with amount

```
SELECT
    s.bill_no,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	qty_sold	price	amount
1001	2	50	100
1002	1	1200	1200
1003	1	2000	2000
1004	3	300	900
1005	2	400	800
1006	1	500	500
1007	1	600	600
1008	4	150	600
1009	1	800	800
1010	5	100	500

5. Customers who bought price > 200

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
2	Neha
3	Rahul
5	Rohit
7	Karan
4	Sneha
6	Priya

6. Count products bought by each customer

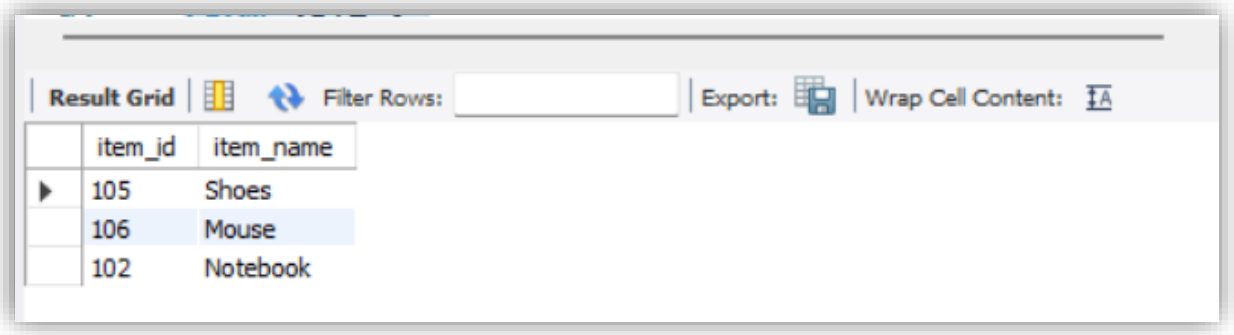
```
SELECT
    c.cust_id,
    c.cust_name,
    SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name	total_products
1	Amit	2
2	Neha	1
3	Rahul	3
4	Sneha	1
5	Rohit	7
6	Priya	1
7	Karan	1
8	Pooja	5

7. Products bought by cust_id = 5

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```



item_id	item_name
105	Shoes
106	Mouse
102	Notebook

8. Item details sold today

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	item_name	price
101	Pen	50
103	Bag	1200
105	Shoes	2000
108	Calculator	500
102	Notebook	150
109	File	100

9. Bill details view

```
CREATE VIEW Bill_Details AS
SELECT
   s.bill_no,
   s.bill_date,
   s.cust_id,
   s.item_id,
   i.price,
   s.qty_sold,
   (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

OUTPUT

```
SELECT * FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-18	1	101	50	2	100
1002	2026-02-18	2	103	1200	1	1200
1003	2026-02-18	5	105	2000	1	2000
1004	2026-02-17	3	104	300	3	900
1005	2026-02-16	5	106	400	2	800
1006	2026-02-18	4	108	500	1	500
1007	2026-02-15	6	110	600	1	600
1008	2026-02-18	5	102	150	4	600
1009	2026-02-12	7	107	800	1	800
1010	2026-02-18	8	109	100	5	500

10. Weekly sales view

```
CREATE VIEW Weekly_Sales AS
SELECT
   bill_date,
   SUM(s.qty_sold * i.price) AS daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
```

OUTPUT

```
SELECT * FROM Weekly_Sales;
```

Result Grid | Filter Rows: | Export:

bill_date	daily_sales
2026-02-18	4900
2026-02-17	900
2026-02-16	800
2026-02-15	600
2026-02-12	800

ALL ACTIONS EXECUTION OUTPUT

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:15:58	CREATE DATABASE Assignment3_DB	1 row(s) affected	0.078 sec
2	12:15:58	USE Assignment3_DB	0 row(s) affected	0.016 sec
3	12:16:12	CREATE DATABASE Assignment3_DB	Error Code: 1007. Can't create database 'assignment3_db'; database exists	0.016 sec
4	12:16:21	CREATE TABLE Customer (cust_id INT PRIMARY KEY, cust_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.140 sec
5	12:16:21	CREATE TABLE Item (item_id INT PRIMARY KEY, item_name VARCHAR(100) NOT NULL, price INT C...	0 row(s) affected	0.032 sec
6	12:16:21	CREATE TABLE Sale (bill_no INT PRIMARY KEY, bill_date DATE NOT NULL, cust_id INT, item_id I...	0 row(s) affected	0.062 sec
7	12:16:41	INSERT INTO Customer VALUES (1,'Amit'), (2,'Neha'), (3,'Rahul'), (4,'Sneha'), (5,'Rohit'), (6,'Priya'), (7,'Karan'), (8...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.031 sec
8	12:16:53	INSERT INTO Item VALUES (101,'Pen',50), (102,'Notebook',150), (103,'Bag',1200), (104,'Bottle',300), (105,'Sho...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.031 sec
9	12:17:07	INSERT INTO Sale VALUES (1001,CURDATE(),1,101,2), (1002,CURDATE(),2,103,1), (1003,CURDATE(),5,105...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
10	12:17:21	SELECT s.bill_no, c.cust_name, s.item_id FROM Sale s JOIN Customer c ON s.cust_id = c.cust_id WHERE s.bill...	6 row(s) returned	0.000 sec / 0.000 sec
11	12:19:47	SELECT s.bill_no, s.qty_sold, i.price, (s.qty_sold * i.price) AS amount FROM Sale s JOIN Item i ON s.it...	10 row(s) returned	0.000 sec / 0.000 sec
12	12:21:03	SELECT DISTINCT c.cust_id, c.cust_name FROM Customer c JOIN Sale s ON c.cust_id = s.cust_id JOIN Item i...	6 row(s) returned	0.015 sec / 0.000 sec
13	12:22:22	SELECT c.cust_id, c.cust_name, SUM(s.qty_sold) AS total_products FROM Customer c JOIN Sale s ON...	8 row(s) returned	0.016 sec / 0.000 sec
14	12:23:14	SELECT i.item_id, i.item_name FROM Sale s JOIN Item i ON s.item_id = i.item_id WHERE s.cust_id = 5 LIMIT 0, ...	3 row(s) returned	0.000 sec / 0.000 sec
15	12:25:01	SELECT DISTINCT i.* FROM Item i JOIN Sale s ON i.item_id = s.item_id WHERE s.bill_date = CURDATE() LIMI...	6 row(s) returned	0.000 sec / 0.000 sec
16	12:25:53	CREATE VIEW Bill_Details AS SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id, i.price, s.qty_s...	0 row(s) affected	0.015 sec
17	12:26:17	SELECT * FROM Bill_Details LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
18	12:27:46	CREATE VIEW Weekly_Sales AS SELECT bill_date, SUM(s.qty_sold * i.price) AS daily_sales FROM Sale ...	0 row(s) affected	0.032 sec
19	12:28:22	SELECT * FROM Weekly_Sales LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec